

Scrap & Sprout 버전 관리 문서 (Version Controll Document) (v1.0)

1. 버전 관리 시스템(VCS) 사용 방법

우리는 세계 표준인 **Git**과 저장소인 **GitHub**를 사용합니다.

[브랜치 전략: Git Flow 맞보기]

현업에서는 하나의 줄기에 대고 코딩하지 않습니다. 용도별로 가지(Branch)를 칩니다.

- **main**: 언제든지 출시(배포) 가능한 가장 안정적인 상태.
- **develop**: 다음 버전을 위해 개발 중인 코드들이 모이는 곳.
- **feature/기능명**: 효은 님의 '인벤토리 수집', 재엽 님의 'DB 연결'처럼 개별 기능을 만드는 곳. 작업 완료 후 **develop**으로 합칩니다(Merge).

[핵심 명령어 가이드]

1. **Pull**: 작업을 시작하기 전, 팀원들이 올린 최신 코드를 내 컴퓨터로 가져옵니다.
2. **Commit**: 내가 작업한 내용을 로컬 기록에 저장합니다. (메모 필수!)
3. **Push**: 내 컴퓨터의 기록을 온라인(GitHub)에 올립니다.
4. **Pull Request (PR)**: "내가 만든 기능을 **develop**에 합쳐도 될까요?"라고 팀원들에게 검토를 요청하는 단계입니다.

2. 코드 작성 규칙 (Commit Message Convention)

변경 이력을 보고 "아, 이때 이걸 고쳤구나!"라고 한눈에 알 수 있게 쓰는 ****'약속'****입니다.

[커밋 메시지 구조]

| 타입: 요약 내용 (예: feat: 아이템 수집 로직 구현)

타입 (Type)	의미	사용 예시
feat	새로운 기능 추가	feat: 쓰레기 수집 및 인벤토리 저장 기능 추가

fix	버그 수정	fix: NPC 판매 시 골드 미증가 오류 수정
docs	문서 수정	docs: 요구사항 명세서 업데이트
style	코드 포맷 변경 (로직 수정 없음)	style: 변수명 명명 규칙에 맞춰 수정
refactor	코드 리팩토링 (가독성/성능 개선)	refactor: 쓰레기 리젠 알고리즘 최적화

3. 코드 변경 이력 (Changelog)

프로젝트가 진행되면서 어떤 큰 변화가 있었는지 기록하는 일지입니다. 나중에 "이 기능 언제 생겼지?"를 찾을 때 필수적이죠.

[Scrap & Sprout 변경 이력 예시]

v0.1.0 (2026-03-29)

- **[Client]** 유니티 2D 쿼터뷰 기본 맵 세팅 및 캐릭터 이동 시스템 구현.
- **[Client]** Trigger 기반 쓰레기 수집 및 인벤토리 디렉터리 시스템 구축.
- **[Network]** Photon Fusion 세션 연결 기본 모듈 작성.
- **[Backend]** Spring Boot 프로젝트 생성 및 MySQL 연동 환경 설정.
- **[AI]** FastAPI 기반 Gemini API 연동 테스트 서버 구축.